DATABASE RELATIONSHIP ANALYSIS USING JOINS

Topic: Flower Shop Management System

Scenario:

A flower shop stores information about customers, flowers/products, categories, sellers, orders, order details, and payments in separate tables. The management team wants to combine related records to understand customer purchases, flower sales, order status, and payment details.

Assumed table relationships

Table	Purpose / key columns
CUSTOMER	CUSTOMER_ID (PK), CUSTOMER_NAME
PRODUCT	PRODUCT_ID (PK), PRODUCT_NAME, CATEGORY_ID, SELLER_ID, PRICE
CATEGORY	CATEGORY_ID (PK), CATEGORY_NAME
SELLER	SELLER_ID (PK), SELLER_NAME
ORDERS	ORDER_ID (PK), CUSTOMER_ID (FK), ORDER_DATE, ORDER_STATUS, TOTAL_AMOUNT
ORDER_DETAILS	ORDER_ID (FK), PRODUCT_ID (FK), QUANTITY
PAYMENT	PAYMENT_ID (PK), ORDER_ID (FK), PAYMENT_MODE, PAYMENT_STATUS, TRANSACTI

N_AMOUN

Note: The SQL examples use these assumed column names. If your actual table has different column names, replace them accordingly.

1. IMPLEMENT INNER JOIN

Concept: INNER JOIN returns only records that have matching values in both tables.

Task 1: Display customer details along with their orders.

```
SQL> SELECT C.Customer_Name, O.Order_ID, O.Total_Amount
  2  FROM Customer C
  3  INNER JOIN Orders O
  4  ON C.Customer_ID = O.Customer_ID;

CUSTOMER_NAME   ORDER_ID   TOTAL_AMOUNT
--------------- ---------- ------------
Ananya          0001             3200.00
Priya           0002             1299.00
Kaviya          0003             1000.00
Karu            0004             1500.00
Gobi            0005              150.00
```

Task 2: Display order details with payment information.

```
SQL> SELECT O.Order_ID, O.Total_Amount, O.Order_Date, O.Customer_ID, P.Payment_Date, P.Payment_Mode, P.Payment_Status FROM Payment P INNER JOIN Orders O ON
P.Order_ID = O.Order_ID;

ORDER_ID   TOTAL_AMOUNT ORDER_DAT CUSTOMER_ID PAYMENT_D PAYMENT_MODE        PAYMENT_STATUS
---------- ------------ --------- ----------- --------- ------------------ --------------------
0001           3200.00 01-SEP-26         101 03-SEP-26 UPI                COMPLETED
0002           1299.00 03-SEP-26         102 04-SEP-26 CARD               FAILED
0003           1000.00 05-SEP-26         103 09-SEP-26 NET BANKING        PENDING
0004           1500.00 23-DEC-26         104 25-DEC-26 CARD               FAILED
0005            150.00 31-JUL-08         105 14-AUG-08 CASH               COMPLETED
```

Task 3: Retrieve flowers/products purchased by customers.

```
SQL> SELECT C.CUSTOMER_NAME, O.ORDER_ID, P.PRODUCT_NAME, OD.QUANTITY, OD.PRICE FROM CUSTOME
R C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD
.ORDER_ID JOIN PRODUCT P ON OD.PRODUCT_ID = P.PRODUCT_ID;

CUSTOMER_NAM ORDER_ID PRODUCT_NAME QUANTITY  PRICE
------------ -------- ------------ -------- ------
Ananya       0001     Red Rose          100   3250
Priya        0002      White Lily        60   1500
Kaviya       0003      Jasmine          150    300
Karu         0004     Purple Orchi       50   2500
                      d

Gobi         0005      Rose Bouque       30    399
                      t
```

2.IMPLEMENT LEFT JOIN

Concept: LEFT JOIN returns all rows from the left table and matching rows from the right table. Unmatched right-side values appear as NULL.

Task 1: Display all customers, including customers who have not placed orders.

```
SQL> select c.customer_id, c.customer_name, o.order_id, o.order_date, o.total_amount
  2  from customer c
  3  left join orders o on c.customer_id = o.customer_id;

CUSTOMER_ID CUSTOMER_NAM ORDER_ID ORDER_DAT TOTAL_AMOUNT
----------- ------------ -------- --------- ------------
        101 Ananya       0001     01-SEP-26         3200
        102 Priya        0002     03-SEP-26         1299
        103 Kaviya       0003     05-SEP-26         1000
        104 Karu         0004     23-DEC-26         1500
        105 Gobi         0005     31-JUL-08          150
```

Task 2: Find customers without any purchases.

```
SQL> SELECT c.customer_id, c.customer_name
  2  FROM customer c
  3  WHERE EXISTS (
  4    SELECT 1
  5    FROM orders o
  6    WHERE o.customer_id = c.customer_id
  7  );

CUSTOMER_ID CUSTOMER_NAM
----------- -------------
        101 Ananya
        102 Priya
        103 Kaviya
        104 Karu
        105 Gobi
```

Task 3: Display all flowers/products, including products with no sales.

```
SQL> SELECT p.product_id, p.product_name, od.order_id, od.quantity, od.price
  2  FROM product p
  3  LEFT JOIN order_details od ON p.product_id = od.product_id;

PRODUCT PRODUCT_NAME ORDER_ID QUANTITY  PRICE
------- ------------ -------- -------- ------
11      Red Rose     0001          100   3250
12       White Lily  0002           60   1500
13       Jasmine     0003          150    300
14      Purple Orchi 0004           50   2500
        d

15       Rose Bouque 0005           30    399
        t
```

2. IMPLEMENT RIGHT JOIN

Concept: RIGHT JOIN returns all rows from the right table and matching rows from the left table.

Task 1: Display all orders with customer information.

```
SQL> SELECT o.order_id, o.order_date, o.total_amount, o.order_status, c.customer_id, c.cust
omer_name
  2  FROM customer c
  3  RIGHT JOIN orders o ON c.customer_id = o.customer_id;

ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         CUSTOMER_ID CUSTOMER_NAM
-------- --------- ------------ -------------------- ----------- ------------
0001     01-SEP-26         3200 Pending                      101 Ananya
0002     03-SEP-26         1299 Confirmed                    102 Priya
0003     05-SEP-26         1000 Ready                        103 Kaviya
0004     23-DEC-26         1500 Delivered                    104 Karu
0005     31-JUL-08          150 Ready                        105 Gobi
```

Task 2: Find orders where customer details are missing.

```
SQL> SELECT o.order_id, o.customer_id, o.order_date
  2  FROM orders o
  3  LEFT JOIN customer c ON o.customer_id = c.customer_id
  4  WHERE c.customer_id IS NULL;

no rows selected
```

Task 3: Display all payment records with order details.

```
SQL> SELECT o.order_id, o.customer_id, o.order_date
  2  FROM orders o
  3  LEFT JOIN customer c ON o.customer_id = c.customer_id
  4  WHERE c.customer_id IS NULL;

no rows selected
```

3. RETRIEVE COMPLETE ORDER DETAILS

Task: Generate a complete flower shop order report containing customer name, flower/product name, quantity, order date, total amount, and payment status.

Query

```
SQL> SELECT c.customer_name, p.product_name, od.quantity, o.order_date, o.total_amount, pay.payment_status
  2  FROM customer c
  3  JOIN orders o ON c.customer_id = o.customer_id
  4  JOIN order_details od ON o.order_id = od.order_id
  5  JOIN product p ON od.product_id = p.product_id
  6  LEFT JOIN payment pay ON o.order_id = pay.order_id;

CUSTOMER_NAM PRODUCT_NAME QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
------------ ------------ -------- --------- ------------ --------------------
Ananya       Red Rose          100 01-SEP-26         3200 COMPLETED
Priya         White Lily        60 03-SEP-26         1299 FAILED
Kaviya        Jasmine          150 05-SEP-26         1000 PENDING
Karu         Purple Orchi       50 23-DEC-26         1500 FAILED
             d

Gobi          Rose Bouque       30 31-JUL-08          150 COMPLETED
             t
```

4. DISPLAY CUSTOMER PURCHASE HISTORY

Task 1: Display all flowers/products purchased by a customer.

```
SQL> SELECT c.customer_name, p.product_name, od.quantity, od.price, o.order_date FROM customer c JOIN orders o ON c.customer_id = o.customer_id JOIN order_d
etails od ON o.order_id = od.order_id JOIN product p ON od.product_id = p.product_id WHERE c.customer_id = '101';

CUSTOMER_NAME   | PRODUCT_NAME    |   QUANTITY |      PRICE | ORDER_DAT
--------------- | --------------- | ---------- | ---------- | ---------
Ananya          | Red Rose        |        100 |       3250 | 01-SEP-26
```

Task 2: Find the total amount spent by each customer.

```
SQL> SELECT c.customer_id,c.customer_name,NVL(SUM(o.total_amount),0) AS total_spent FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GROU
P BY c.customer_id,c.customer_name;

CUSTOMER_ID | CUSTOMER_NAME   | TOTAL_SPENT
----------- | --------------- | -----------
        102 | Priya           |        1299
        104 | Karu            |        1500
        105 | Gobi            |         150
        101 | Ananya          |        3200
        103 | Kaviya          |        1000
```

Task 3: Find the number of orders placed by each customer.

```
SQL> SELECT c.customer_id,c.customer_name,COUNT(o.order_id) AS total_orders FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.c
ustomer_id,c.customer_name;

CUSTOMER_ID | CUSTOMER_NAME   | TOTAL_ORDERS
----------- | --------------- | ------------
        102 | Priya           |            1
        104 | Karu            |            1
        105 | Gobi            |            1
        101 | Ananya          |            1
        103 | Kaviya          |            1
```

Task 4: Display the latest purchase details of customers.

```
SQL> SELECT c.customer_name, o.order_id, o.order_date, o.total_amount, o.order_status FROM customer c JOIN orders o ON c.customer_id = o.customer_id WHERE o
.order_date = (SELECT MAX(o2.order_date) FROM orders o2 WHERE o2.customer_id = c.customer_id);

CUSTOMER_NAME   | ORDER_ID   | ORDER_DAT | TOTAL_AMOUNT | ORDER_STATUS
--------------- | ---------- | --------- | ------------ | --------------------
Ananya          | 0001       | 01-SEP-26 |         3200 | Pending
Priya           | 0002       | 03-SEP-26 |         1299 | Confirmed
Kaviya          | 0003       | 05-SEP-26 |         1000 | Ready
Karu            | 0004       | 23-DEC-26 |         1500 | Delivered
Gobi            | 0005       | 31-JUL-08 |          150 | Ready
```

5. GENERATE MULTI-TABLE BUSINESS REPORTS

Report 1: Customer Order Report

Display customer name, order ID, order date, and order status.

```
SQL> SELECT c.customer_name, o.order_id, o.order_date,o.order_status FROM c
stomer c JOIN orders o ON  c.customer_id = o.customer_id;

CUSTOMER_NAME   | ORDER_ID   | ORDER_DAT | ORDER_STATUS
--------------- | ---------- | --------- | --------------------
Ananya          | 0001       | 01-SEP-26 | Pending
Priya           | 0002       | 03-SEP-26 | Confirmed
Kaviya          | 0003       | 05-SEP-26 | Ready
Karu            | 0004       | 23-DEC-26 | Delivered
Gobi            | 0005       | 31-JUL-08 | Ready
```

Report 2: Flower/Product Sales Report

Display product name, quantity sold, and total revenue.

```
SQL> SELECT c.customer_name, o.order_id, o.order_date,o.order_status FROM customer c JOIN orders o ON  c.customer_id = o.customer_i

CUSTOMER_NAME   | ORDER_ID   | ORDER_DAT | ORDER_STATUS
--------------- | ---------- | --------- | --------------------
Ananya          | 0001       | 01-SEP-26 | Pending
Priya           | 0002       | 03-SEP-26 | Confirmed
Kaviya          | 0003       | 05-SEP-26 | Ready
Karu            | 0004       | 23-DEC-26 | Delivered
Gobi            | 0005       | 31-JUL-08 | Ready
```

Report 3: Payment Analysis Report

Display payment mode, number of transactions, and successful payments.

```
SQL> SELECT p.product_name, SUM(od.quantity) AS quantity_sold, SUM(od.quantity * od.price) AS total_revenue FROM product p JOIN order_details od ON p.pro
t_id = od.product_id GROUP BY p.product_id, p.product_name;

PRODUCT_NAME    | QUANTITY_SOLD | TOTAL_REVENUE
--------------- | ------------- | -------------
 Jasmine        |           150 |         45000
 Rose Bouquet   |            30 |         11970
Red Rose        |           100 |        325000
 White Lily     |            60 |         90000
Purple Orchid   |            50 |        125000
```

Report 4: Customer Purchase Analysis

Find customers with the highest spending.

```
SQL> SELECT payment_mode, COUNT(payment_id) AS total_transactions, SUM(CASE WHEN payment_status = 'Success' THEN 1 EL
0 END) AS successful_payments FROM payment GROUP BY payment_mode;

PAYMENT_MODE        | TOTAL_TRANSACTIONS | SUCCESSFUL_PAYMENTS
------------------- | ------------------ | -------------------
CARD                |                  2 |                   0
NET BANKING         |                  1 |                   0
CASH                |                  1 |                   0
UPI                 |                  1 |                   0
```

Additional query: Customers with the maximum number of orders

```
SQL> SELECT c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.cust
mer_id GROUP BY c.customer_id, c.customer_name ORDER BY total_spent DESC;

CUSTOMER_NAME   | TOTAL_SPENT
--------------- | -----------
Ananya          |        3200
Karu            |        1500
Priya           |        1299
Kaviya          |        1000
Gobi            |         150
```

```
SQL> SELECT c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.cust
mer_id GROUP BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) = (SELECT MAX(customer_spending) FROM (SELECT
SUM(total_amount) AS customer_spending FROM orders GROUP BY customer_id));

CUSTOMER_NAME   | TOTAL_SPENT
--------------- | -----------
Ananya          |        3200
```

```
SQL> SELECT c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.custo
er_id GROUP BY c.customer_id, c.customer_name HAVING COUNT(o.order_id) = (SELECT MAX(order_count) FROM (SELECT COUNT(*)
AS order_count FROM orders GROUP BY customer_id));

CUSTOMER_NAME   | TOTAL_ORDERS
--------------- | ------------
Priya           |            1
Karu            |            1
Gobi            |            1
Ananya          |            1
Kaviya          |            1
```

```
SQL> SELECT c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.cust
mer_id GROUP BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) = (SELECT MAX(customer_spending) FROM (SELECT
SUM(total_amount) AS customer_spending FROM orders GROUP BY customer_id));

CUSTOMER_NAME   | TOTAL_SPENT
--------------- | -----------
Ananya          |        3200
```